

Main Code:

```
package com.example.blackjack;
import javafx.animation.FadeTransition;
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.*;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.scene.text.FontPosture;
import javafx.scene.text.FontWeight;
import javafx.stage.Stage;
import javafx.util.Duration;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.*;

import static javafx.animation.Animation.INDEFINITE;

public class Blackjack extends Application {
    Label title, house, lwin, llose, congratulations, enterHighScore, highScore,
    lName, lScore, Num1,
    Num2, Num3, Num4, Num5, Num6, Num7, Num8, Num9, Num10, player, bust,
    gameWin, gameLose, push, blackJack, lgameOver;
    Button titleStart, titleHighScore, exit, hit, stand, gameContinue,
    highScoreEnterContinue, highScore1Next,
    highScore2Prev, backToTitle, play;
    TextField enterHighScoreBox, name0, score0, name1, score1,
    name2, score2, name3, score3, name4, score4, name5, score5, name6,
    score6, name7, score7, name8,
    score8, name9, score9, playerScore, houseScore, win, lose, winRec,
    loseRec;
    int winCounter = 0 , loseCounter = 0, playerHand,houseHand, playerCardCount,
    houseCardCount, cardNum, playerAdd, houseAdd, pageCount,newWin,newLose, k = 0,
    rulesSeen = 0;
    boolean gameInProgress, gameOver;
    Stage myStage;
    MyArrayList records =new MyArrayList();
    ArrayList<HighScores> scores = new ArrayList<>();
    ImageView playerCard1, playerCard2, playerCard3, playerCard4, playerCard5,
    houseCard1, houseCard2, houseCard3, houseCard4, houseCard5;
    Image p1, p2, p3, p4, p5, h1, h2, h3, h4, h5, tp;
```

```

@Override
public void start(Stage stage) throws IOException {
    myStage = stage;
    stage.setTitle("Blackjack");
    if (k<=1) {
        Records();
        k++;
    }
    stage.setScene(titleScene());
    highScores();
    stage.show();
}

public ArrayList<HighScores> highScores(){
    ArrayList<String> list=MyFile.readFile("highscores.csv");
    for (String info : list) {
        String[] fields = info.split(",");
        HighScores score = new HighScores(fields[0].trim(),
Integer.parseInt(fields[1].trim()));
        scores.add(score);
    }
    return scores;
}

public MyArrayList Records() {
    ArrayList<String> list = MyFile.readFile("Records.csv");
    for (String info : list) {
        String[] fields = info.split(",");
        Records record = new Records(Integer.parseInt(fields[0].trim()));
        records.add(record);
    }
    return records;
}

public void saveRecords(){
    Records rec = (Records) records.get(0);
    int prevWin = rec.getWinRecord();
    newWin = prevWin + winCounter;
    records.remove(0);
    records.add(new Records(newWin));
    MyFile.writeFile1(records, "Records.csv");
}

public Scene titleScene() throws FileNotFoundException {
    saveRecords();
    gameOver = false;
    winCounter = 0;
    loseCounter = 0;
    BorderPane root = new BorderPane();
    tp = new Image((new FileInputStream("pics/blackjack.png")));
    title = new Label("Blackjack");
    title.setFont(Font.font("Brush Script MT", FontWeight.BOLD,
FontPosture.ITALIC, 60));

```

```

title.setTextFill(Color.GOLDENROD);
FadeTransition fade = new FadeTransition();
fade.setDuration(Duration.millis(1500));
fade.setFromValue(10);
fade.setToValue(0.1);
fade.setCycleCount(INDEFINITE);
fade.setAutoReverse(true);
fade.setNode(title);
fade.play();
title.setLayoutX(150);
title.setLayoutY(100);
Label winRecLabel = new Label("Total Wins:");
winRec = new TextField();
winRec.setEditable(false);
winRec.setMaxSize(30,20);
winRec.setText(String.valueOf(newWin));
ImageView titlePic = new ImageView();
titlePic.setImage(tp);
titlePic.setFitHeight(200);
titlePic.setFitWidth(300);
titleStart = new Button("Start Game");
titleHighScore = new Button("High Scores");
exit = new Button("Exit");
titleStart.setOnAction(e->{
    if (rulesSeen == 0) {
        myStage.setScene(rulesScene());
    }
    else {
        myStage.setScene(gameScene());
    }
});
titleHighScore.setOnAction(e->{
    myStage.setScene(highScorePage1Scene());
});
exit.setOnAction(e->{
    MyFile.writeFile(scores,"highscores.csv");
    System.exit(0);
});
HBox buttons = new HBox();
HBox title1 = new HBox();
title1.setAlignment(Pos.TOP_CENTER);
title1.getChildren().addAll(title);
buttons.setSpacing(5);
buttons.setAlignment(Pos.CENTER);
buttons.getChildren().addAll(winRecLabel, winRec, titleStart,
titleHighScore, exit);
root.setTop(title1);
root.setCenter(titlePic);
root.setBottom(buttons);

```

```

        return new Scene(root, 450, 400, Color.BLACK);
    }
    public Scene rulesScene(){
        rulesSeen++;
        BorderPane root = new BorderPane();
        title = new Label("Rules of The Game");
        title.setFont(Font.font("Brush Script MT", FontWeight.BOLD,
FontPosture.ITALIC, 60));
        title.setTextFill(Color.GOLDENROD);
        FadeTransition fade = new FadeTransition();
        fade.setDuration(Duration.millis(1500));
        fade.setFromValue(10);
        fade.setToValue(0.1);
        fade.setCycleCount(INDEFINITE);
        fade.setAutoReverse(true);
        fade.setNode(title);
        fade.play();
        Label rules = new Label("Hello and welcome to Blackjack, one of the
world's most famous card games!!!");
        Label rules2 = new Label("The rules of this game is quite simple: you
(the player) have to get as close");
        Label rules3 = new Label("to 21 as you can without going over or having
a score lower than the house!!!");
        Label rules4 = new Label("Your goal is to win as many times as you can
before you lose three games!!!");
        Label rules5 = new Label("Remember if you get Blackjack, you'll earn two
points instead of one!!!");
        Label rules6 = new Label("Have Fun!!!");
        Button startGame = new Button("Continue");
        startGame.setOnAction(e->{
            myStage.setScene(gameScene());
        });
        VBox info = new VBox();
        info.setSpacing(10);
        info.setAlignment(Pos.CENTER);
        info.getChildren().addAll(rules, rules2, rules3, rules4, rules5, rules6);
        HBox button = new HBox(); HBox title1 = new HBox();
        title1.setAlignment(Pos.TOP_CENTER);
        title1.getChildren().addAll(title);
        button.setAlignment(Pos.BOTTOM_CENTER);
        button.getChildren().addAll(startGame);
        root.setTop(title1);
        root.setCenter(info);
        root.setBottom(button);
        return new Scene(root, 500, 600);
    }
    public Scene gameScene(){
        BorderPane root = new BorderPane();
        GridPane leftGrid = new GridPane();

```

```

GridPane rightGrid = new GridPane();
playerCard1 = new ImageView();
playerCard1.setFitWidth(100);
playerCard1.setFitHeight(120);
playerCard2 = new ImageView();
playerCard2.setFitWidth(100);
playerCard2.setFitHeight(120);
playerCard3 = new ImageView();
playerCard3.setFitWidth(100);
playerCard3.setFitHeight(120);
playerCard4 = new ImageView();
playerCard4.setFitWidth(100);
playerCard4.setFitHeight(120);
playerCard5 = new ImageView();
playerCard5.setFitWidth(100);
playerCard5.setFitHeight(120);
houseCard1 = new ImageView();
houseCard1.setFitWidth(100);
houseCard1.setFitHeight(120);
houseCard2 = new ImageView();
houseCard2.setFitWidth(100);
houseCard2.setFitHeight(120);
houseCard3 = new ImageView();
houseCard3.setFitWidth(100);
houseCard3.setFitHeight(120);
houseCard4 = new ImageView();
houseCard4.setFitWidth(100);
houseCard4.setFitHeight(120);
houseCard5 = new ImageView();
houseCard5.setFitWidth(100);
houseCard5.setFitHeight(120);
leftGrid.setVgap(5);
rightGrid.setVgap(5);
house = new Label("House Score: " );
player = new Label("Player Score: " );
lwin = new Label("Wins: " );
llose = new Label("Losses: " );
bust = new Label("BUST");
bust.setFont(Font.font("Verdana", FontWeight.BOLD,24));
bust.setTextFill(Color.RED);
gameLose = new Label("YOU LOSE");
gameLose.setFont(Font.font("Verdana", FontWeight.BOLD,24));
gameLose.setTextFill(Color.RED);
gameWin = new Label("YOU WIN");
gameWin.setFont(Font.font("Verdana", FontWeight.BOLD,24));
gameWin.setTextFill(Color.GREEN);
push = new Label("PUSH");
push.setFont(Font.font("Verdana", FontWeight.BOLD,24));
blackJack = new Label("BLACKJACK");

```

```

blackJack.setFont(Font.font("Verdana", FontWeight.BOLD, 24));
blackJack.setTextFill(Color.GOLDENROD);
lgameOver = new Label("GAME OVER");
lgameOver.setFont(Font.font("Verdana", FontWeight.BOLD, 28));
lgameOver.setTextFill(Color.RED);
FadeTransition fade = new FadeTransition();
fade.setDuration(Duration.millis(750));
fade.setFromValue(10);
fade.setToValue(0.1);
fade.setCycleCount(INDEFINITE);
fade.setAutoReverse(true);
fade.setNode(lgameOver);
fade.play();
bust.setVisible(false);
gameWin.setVisible(false);
gameLose.setVisible(false);
push.setVisible(false);
blackJack.setVisible(false);
lgameOver.setVisible(false);
playerScore = new TextField();
playerScore.setMaxSize(30, 10);
playerScore.setEditable(false);
houseScore = new TextField();
houseScore.setMaxSize(30, 10);
houseScore.setEditable(false);
win = new TextField();
win.setMaxSize(30, 10);
win.setEditable(false);
lose = new TextField();
lose.setMaxSize(30, 10);
lose.setEditable(false);
HBox buttons = new HBox();
hit = new Button("Hit");
stand = new Button("Stand");
gameContinue = new Button("Continue");
play = new Button("Play");
exit = new Button("Exit");
backToTitle = new Button("Back to Title");
hit.setOnAction(e->{
    try {
        player();
    } catch (FileNotFoundException ex) {
        throw new RuntimeException(ex);
    }
});
stand.setOnAction(e->{
    gameInProgress = false;
    try {
        houseCont();
    }
});

```

```

        } catch (FileNotFoundException ex) {
            throw new RuntimeException(ex);
        }

    });
    play.setOnAction(e-> {
        endGame();
        hideGameStats();
        gameInProgress = true;
        try {
            player();
        } catch (FileNotFoundException ex) {
            throw new RuntimeException(ex);
        }
    });
    backToTitle.setOnAction(e -> {
        gameInProgress = false;
        try {
            myStage.setScene(titleScene());
        } catch (FileNotFoundException ex) {
            throw new RuntimeException(ex);
        }
    });
    gameContinue.setOnAction(e-> {
        sortByScore();
        int compareScore = scores.get(19).getScore();
        if(winCounter > compareScore) {
            myStage.setScene(highScoreEntryScene());
        }
        else{
            try {
                myStage.setScene(titleScene());
            } catch (FileNotFoundException ex) {
                throw new RuntimeException(ex);
            }
        }
    });
    exit.setOnAction(e-> {
        MyFile.writeFile(scores, "highscores.csv");
        System.exit(0);
    });
    gameLogic();
    win.setText(String.valueOf(winCounter));
    lose.setText(String.valueOf(loseCounter));
    buttons.setSpacing(10);
    buttons.setAlignment(Pos.CENTER);
    buttons.getChildren().addAll(lwin,win,hit, stand, play, gameContinue,
backToTitle, exit, llose,lose);
    StackPane gameEnd = new StackPane();

```

```

        gameEnd.getChildren().addAll(bust, gameWin, gameLose, push, blackJack,
lgameOver);
        gameEnd.setAlignment(Pos.BOTTOM_CENTER);
        leftGrid.addColumn(0, player, playerScore, playerCard1, playerCard2,
playerCard3, playerCard4, playerCard5);
        rightGrid.addColumn(0, house, houseScore, houseCard1, houseCard2,
houseCard3, houseCard4, houseCard5);
        root.setLeft(leftGrid);
        root.setRight(rightGrid);
        root.setBottom(buttons);
        root.setCenter(gameEnd);

        return new Scene(root, 600,700);
    }
    public Scene highScoreEntryScene(){
        BorderPane root = new BorderPane();
        GridPane pane = new GridPane();
        congratulations = new Label("CONGRATULATIONS!!!");
        congratulations.setFont(Font.font("Brush Script MT", FontWeight.BOLD,
FontPosture.ITALIC, 32));
        congratulations.setTextFill(Color.GOLDENROD);
        FadeTransition fade = new FadeTransition();
        fade.setDuration(Duration.millis(1000));
        fade.setFromValue(10);
        fade.setToValue(0.1);
        fade.setCycleCount(INDEFINITE);
        fade.setAutoReverse(true);
        fade.setNode(congratulations);
        fade.play();
        enterHighScore = new Label("You got a high score!!!
Enter your name!!!");
        enterHighScore.setTextFill(Color.GOLDENROD);
        enterHighScoreBox = new TextField();
        highScoreEnterContinue = new Button("Continue");
        HBox buttons = new HBox();
        buttons.setAlignment(Pos.CENTER);
        buttons.getChildren().addAll(highScoreEnterContinue);
        pane.addRow(0, congratulations);
        pane.addRow(1, enterHighScore);
        pane.addRow(2, enterHighScoreBox);
        pane.setHgap(10);
        pane.setAlignment(Pos.CENTER);
        root.setCenter(pane);
        root.setBottom(buttons);
        highScoreEnterContinue.setOnAction(e->{
            String addName = enterHighScoreBox.getText();
            scores.add(new HighScores(addName,winCounter));
            int compareScore = scores.get(9).getScore();
            if(winCounter > compareScore) {

```

```

        myStage.setScene(highScorePage1Scene());
    }else {
        myStage.setScene(highScorePage2Scene());
    }
});
return new Scene(root, 440,450, Color.MEDIUMSPRINGGREEN);
}
public Scene highScorePage1Scene(){
    pageCount = 1;
    BorderPane root = new BorderPane();
    GridPane topPane = new GridPane();
    GridPane pane = new GridPane();
    highScore = new Label("High Scores");
    highScore.setTextFill(Color.GOLDENROD);
    highScore.setFont(Font.font("Brush Script MT", FontWeight.BOLD,
FontPosture.ITALIC, 32));
    highScore.setTextFill(Color.GOLDENROD);
    FadeTransition fade = new FadeTransition();
    fade.setDuration(Duration.millis(1000));
    fade.setFromValue(10);
    fade.setToValue(0.1);
    fade.setCycleCount(INDEFINITE);
    fade.setAutoReverse(true);
    fade.setNode(highScore);
    fade.play();
    lName = new Label("Name");
    lScore = new Label("Score");
    Num1 = new Label("1:");
    Num1.setTextFill(Color.GOLDENROD);
    Num2 = new Label("2:");
    Num2.setTextFill(Color.SLATEGRAY);
    Num3 = new Label("3:");
    Num3.setTextFill(Color.BROWN);
    Num4 = new Label("4:");
    Num5 = new Label("5:");
    Num6 = new Label("6:");
    Num7 = new Label("7:");
    Num8 = new Label("8:");
    Num9 = new Label("9:");
    Num10 = new Label("10:");
    name0 = new TextField();
    name0.setEditable(false);
    score0 = new TextField();
    score0.setEditable(false);
    name1 = new TextField();
    name1.setEditable(false);
    score1 = new TextField();
    score1.setEditable(false);
    name2 = new TextField();

```

```

name2.setEditable(false);
score2 = new TextField();
score2.setEditable(false);
name3 = new TextField();
name3.setEditable(false);
score3 = new TextField();
score3.setEditable(false);
name4 = new TextField();
name4.setEditable(false);
score4 = new TextField();
score4.setEditable(false);
name5 = new TextField();
name5.setEditable(false);
score5 = new TextField();
score5.setEditable(false);
name6 = new TextField();
name6.setEditable(false);
score6 = new TextField();
score6.setEditable(false);
name7 = new TextField();
name7.setEditable(false);
score7 = new TextField();
score7.setEditable(false);
name8 = new TextField();
name8.setEditable(false);
score8 = new TextField();
score8.setEditable(false);
name9 = new TextField();
name9.setEditable(false);
score9 = new TextField();
score9.setEditable(false);
backToTitle = new Button("Back to Title Screen");
highScore1Next = new Button("Next");
exit = new Button("Exit");
HBox buttons = new HBox();
buttons.setSpacing(5);
buttons.setAlignment(Pos.CENTER);
buttons.getChildren().addAll(highScore1Next, backToTitle, exit);
topPane.addRow(0, highScore);
topPane.addRow(1, lName, lScore);
topPane.setVgap(20);
topPane.setAlignment(Pos.TOP_CENTER);
pane.addRow(0, Num1, name0, score0);
pane.addRow(1, Num2, name1, score1);
pane.addRow(2, Num3, name2, score2);
pane.addRow(3, Num4, name3, score3);
pane.addRow(4, Num5, name4, score4);
pane.addRow(5, Num6, name5, score5);
pane.addRow(6, Num7, name6, score6);

```

```

pane.addRow(7, Num8, name7, score7);
pane.addRow(8, Num9, name8, score8);
pane.addRow(9, Num10, name9, score9);
pageLoader();
backToTitle.setOnAction(e -> {
    try {
        myStage.setScene(titleScene());
    } catch (FileNotFoundException ex) {
        throw new RuntimeException(ex);
    }
});
highScore1Next.setOnAction(e->{
    myStage.setScene(highScorePage2Scene());
});
exit.setOnAction(e-> {
    MyFile.writeFile(scores, "highscores.csv");
    System.exit(0);
});
root.setTop(topPane);
root.setCenter(pane);
root.setBottom(buttons);
return new Scene(root, 340, 400, Color.MEDIUMSPRINGGREEN);
}

public Scene highScorePage2Scene() {
    pageCount = 2;
    BorderPane root = new BorderPane();
    GridPane topPane = new GridPane();
    GridPane pane = new GridPane();
    highScore = new Label("High Scores");
    highScore.setTextFill(Color.GOLDENROD);
    highScore.setFont(Font.font("Brush Script MT", FontWeight.BOLD,
FontPosture.ITALIC, 32));
    highScore.setTextFill(Color.GOLDENROD);
    FadeTransition fade = new FadeTransition();
    fade.setDuration(Duration.millis(1000));
    fade.setFromValue(10);
    fade.setToValue(0.1);
    fade.setCycleCount(INDEFINITE);
    fade.setAutoReverse(true);
    fade.setNode(highScore);
    fade.play();
    lName = new Label("Name");
    lScore = new Label("Score");
    Num1 = new Label("11:");
    Num2 = new Label("12:");
    Num3 = new Label("13:");
    Num4 = new Label("14:");
    Num5 = new Label("15:");
    Num6 = new Label("16:");

```

```
Num7 = new Label("17:");
Num8 = new Label("18:");
Num9 = new Label("19:");
Num10 = new Label("20:");
name0 = new TextField();
name0.setEditable(false);
score0 = new TextField();
score0.setEditable(false);
name1 = new TextField();
name1.setEditable(false);
score1 = new TextField();
score1.setEditable(false);
name2 = new TextField();
name2.setEditable(false);
score2 = new TextField();
score2.setEditable(false);
name3 = new TextField();
name3.setEditable(false);
score3 = new TextField();
score3.setEditable(false);
name4 = new TextField();
name4.setEditable(false);
score4 = new TextField();
score4.setEditable(false);
name5 = new TextField();
name5.setEditable(false);
score5 = new TextField();
score5.setEditable(false);
name6 = new TextField();
name6.setEditable(false);
score6 = new TextField();
score6.setEditable(false);
name7 = new TextField();
name7.setEditable(false);
score7 = new TextField();
score7.setEditable(false);
name8 = new TextField();
name8.setEditable(false);
score8 = new TextField();
score8.setEditable(false);
name9 = new TextField();
name9.setEditable(false);
score9 = new TextField();
score9.setEditable(false);
backToTitle = new Button("Back to Title Screen");
highScore2Prev = new Button("Prev");
exit = new Button("Exit");
HBox buttons = new HBox();
buttons.setSpacing(5);
```

```

        buttons.setAlignment(Pos.CENTER);
        buttons.getChildren().addAll(highScore2Prev, backToTitle, exit);
        topPane.addRow(0, highScore);
        topPane.addRow(1, lName, lScore);
        topPane.setVgap(20);
        topPane.setAlignment(Pos.TOP_CENTER);
        pane.addRow(0, Num1, name0, score0);
        pane.addRow(1, Num2, name1, score1);
        pane.addRow(2, Num3, name2, score2);
        pane.addRow(3, Num4, name3, score3);
        pane.addRow(4, Num5, name4, score4);
        pane.addRow(5, Num6, name5, score5);
        pane.addRow(6, Num7, name6, score6);
        pane.addRow(7, Num8, name7, score7);
        pane.addRow(8, Num9, name8, score8);
        pane.addRow(9, Num10, name9, score9);
        pageLoader();
        backToTitle.setOnAction(e -> {
            try {
                myStage.setScene(titleScene());
            } catch (FileNotFoundException ex) {
                throw new RuntimeException(ex);
            }
        });
        highScore2Prev.setOnAction(e->{
            myStage.setScene(highScorePage1Scene());
        });
        exit.setOnAction(e-> {
            MyFile.writeFile(scores, "highscores.csv");
            System.exit(0);
        });
        root.setTop(topPane);
        root.setCenter(pane);
        root.setBottom(buttons);
        return new Scene(root, 340, 400, Color.MEDIUMSPRINGGREEN);
    }
}

public void pageLoader() {
    sortByScore();
    if (pageCount == 1) {
        int i = 0;
        HighScores HS = scores.get(i);
        name0.setText(HS.getName());
        score0.setText(String.valueOf(HS.getScore()));
        i = 1;
        HighScores HS1 = scores.get(i);
        name1.setText(HS1.getName());
        score1.setText(String.valueOf(HS1.getScore()));
        i = 2;
        HighScores HS2 = scores.get(i);
    }
}

```

```

        name2.setText(HS2.getName());
        score2.setText(String.valueOf(HS2.getScore()));
        i = 3;
        HighScores HS3 = scores.get(i);
        name3.setText(HS3.getName());
        score3.setText(String.valueOf(HS3.getScore()));
        i = 4;
        HighScores HS4 = scores.get(i);
        name4.setText(HS4.getName());
        score4.setText(String.valueOf(HS4.getScore()));
        i = 5;
        HighScores HS5 = scores.get(i);
        name5.setText(HS5.getName());
        score5.setText(String.valueOf(HS5.getScore()));
        i = 6;
        HighScores HS6 = scores.get(i);
        name6.setText(HS6.getName());
        score6.setText(String.valueOf(HS6.getScore()));
        i = 7;
        HighScores HS7= scores.get(i);
        name7.setText(HS7.getName());
        score7.setText(String.valueOf(HS7.getScore()));
        i = 8;
        HighScores HS8= scores.get(i);
        name8.setText(HS8.getName());
        score8.setText(String.valueOf(HS8.getScore()));
        i = 9;
        HighScores HS9= scores.get(i);
        name9.setText(HS9.getName());
        score9.setText(String.valueOf(HS9.getScore()));
    }
    else{
        int i = 10;
        HighScores HS = scores.get(i);
        name0.setText(HS.getName());
        score0.setText(String.valueOf(HS.getScore()));
        i = 11;
        HighScores HS1 = scores.get(i);
        name1.setText(HS1.getName());
        score1.setText(String.valueOf(HS1.getScore()));
        i = 12;
        HighScores HS2 = scores.get(i);
        name2.setText(HS2.getName());
        score2.setText(String.valueOf(HS2.getScore()));
        i = 13;
        HighScores HS3 = scores.get(i);
        name3.setText(HS3.getName());
        score3.setText(String.valueOf(HS3.getScore()));
        i = 14;
    }
}

```

```

        HighScores HS4 = scores.get(i);
        name4.setText(HS4.getName());
        score4.setText(String.valueOf(HS4.getScore()));
        i = 15;
        HighScores HS5 = scores.get(i);
        name5.setText(HS5.getName());
        score5.setText(String.valueOf(HS5.getScore()));
        i = 16;
        HighScores HS6 = scores.get(i);
        name6.setText(HS6.getName());
        score6.setText(String.valueOf(HS6.getScore()));
        i = 17;
        HighScores HS7= scores.get(i);
        name7.setText(HS7.getName());
        score7.setText(String.valueOf(HS7.getScore()));
        i = 18;
        HighScores HS8= scores.get(i);
        name8.setText(HS8.getName());
        score8.setText(String.valueOf(HS8.getScore()));
        i = 19;
        HighScores HS9= scores.get(i);
        name9.setText(HS9.getName());
        score9.setText(String.valueOf(HS9.getScore()));
    }
}

public void gameLogic(){
    if (!gameInProgress){
        play.setDisable(false);
        backToTitle.setDisable(false);
        hit.setDisable(true);
        stand.setDisable(true);
        gameContinue.setDisable(true);
    }
    else {
        play.setDisable(true);
        backToTitle.setDisable(true);
        hit.setDisable(false);
        stand.setDisable(false);
        gameContinue.setDisable(true);
    }
    if (gameOver){
        play.setDisable(true);
        backToTitle.setDisable(true);
        hit.setDisable(true);
        stand.setDisable(true);
        gameContinue.setDisable(false);
    }
}

public void player() throws FileNotFoundException {

```

```

gameLogic();
if (playerCardCount == 0) {
    house();
    //drawing the playing card
    String player1 = toString();
    //getting the value of that playing card
    playerAdd = cardVal();
    //getting image of card
    p1 = new Image((new FileInputStream("pics/"+player1+".png")));
    //setting image of card
    playerCard1.setImage(p1);
    //adding the value of the card the player's score
    playerHand = playerAdd;
    //repeating the process for the second player's card
    String player2 = toString();
    int tempScore = cardVal();
    //getting image of card
    p2 = new Image((new FileInputStream("pics/"+player2+".png")));
    //setting image of card
    playerCard2.setImage(p2);
    playerHand = playerHand + tempScore;
    playerScore.setText(String.valueOf(playerHand));
    playerCardCount = 2;
    if (playerHand == 21) {
        blackJack.setVisible(true);
        houseCont();
    }
} else if (playerCardCount == 2) {
    String player3 = toString();
    int addScore = cardVal();
    if (addScore == 11){
        if (playerHand >= 11){
            addScore = addScore - 10;
        }
    }
    //getting image of card
    p3 = new Image((new FileInputStream("pics/"+player3+".png")));
    //setting image of card;
    playerCard3.setImage(p3);
    playerScore.setText(String.valueOf(playerHand + addScore));
    playerHand = playerHand + addScore;
    playerCardCount++;
    if (playerHand == 21) {
        houseCont();
    } else if (playerHand > 21){
        houseCont();
    }
}
} else if (playerCardCount == 3){

```

```

        String player4 = toString();
        int addScore = cardVal();
        if (addScore == 11){
            if (playerHand >= 11){
                addScore = addScore - 10;
            }
        }
        //getting image of card
        p4 = new Image((new FileInputStream("pics/"+player4+".png")));
        //setting image of card
        playerCard4.setImage(p4);
        playerScore.setText(String.valueOf(playerHand + addScore));
        playerHand = playerHand + addScore;
        playerCardCount++;
        if (playerHand == 21) {
            houseCont();
        } else if (playerHand > 21){
            houseCont();
        }
    }
else if (playerCardCount == 4){
    String player5 = toString();
    int addScore = cardVal();
    if (addScore == 11){
        if (playerHand >= 11){
            addScore = addScore - 10;
        }
    }
    //getting image of card
    p5= new Image((new FileInputStream("pics/"+player5+".png")));
    //setting image of card
    playerCard5.setImage(p5);
    playerScore.setText(String.valueOf(playerHand + addScore));
    playerHand = playerHand + addScore;
    playerCardCount++;
    if (playerHand == 21) {
        houseCont();
    } else if (playerHand > 21){
        houseCont();
    }
}

}

public void house() throws FileNotFoundException {
    gameLogic();
    if (houseCardCount == 0) {
        String house1 = toString();
        houseAdd = cardVal();
        //getting image of card

```

```

        h1 = new Image((new FileInputStream("pics/"+house1+".png")));
        //setting image of card
        houseCard1.setImage(h1);
        houseHand = houseAdd;
        h2 = new Image((new FileInputStream("pics/HIDDEN CARD.png")));
        houseCard2.setImage(h2);
        houseScore.setText(String.valueOf(houseHand));
        houseCardCount = 1;
    }
}

public void houseCont() throws FileNotFoundException {
    if (houseCardCount == 1) {
        String house2 = toString();
        houseAdd = cardVal();
        //getting image of card
        h2 = new Image((new FileInputStream("pics/"+house2+".png")));
        //setting image of card;
        //setting image of card
        houseCard2.setImage(h2);
        houseHand = houseHand + houseAdd;
        houseScore.setText(String.valueOf(houseHand));
        houseCardCount++;
    }
    if (houseHand >= 17) {
        results();

    } else {
        houseCont2();
    }
}

public void houseCont2() throws FileNotFoundException {
    if (houseCardCount == 2 && houseHand <= 16) {
        String house3 = toString();
        houseAdd = cardVal();
        if (houseAdd == 11){
            if (houseHand >= 11){
                houseAdd = houseAdd - 10;
            }
        }
        //getting image of card
        h3 = new Image((new FileInputStream("pics/"+house3+".png")));
        //setting image of card;
        //setting image of card
        houseCard3.setImage(h3);
        houseHand = houseHand + houseAdd;
        houseScore.setText(String.valueOf(houseHand));
        houseCardCount++;
    }
    if (houseHand >= 17) {

```

```

        results();
    }else{
        houseCont3();
    }
}
public void houseCont3 () throws FileNotFoundException {
    if (houseCardCount == 3 && houseHand <= 16) {
        String house4 = toString();
        houseAdd = cardVal();
        if (houseAdd == 11){
            if (houseHand >= 11){
                houseAdd = houseAdd - 10;
            }
        }
        //getting image of card
        h4 = new Image((new FileInputStream("pics/"+house4+".png")));
        //setting image of card;
        //setting image of card
        houseCard4.setImage(h4);
        houseHand = houseHand + houseAdd;
        houseScore.setText(String.valueOf(houseHand));
        houseCardCount++;
    }
    if (houseHand >= 17) {
        results();
    }else{
        houseCont4();
    }
}
public void houseCont4 () throws FileNotFoundException {
    if (houseCardCount == 4 && houseHand <= 16) {
        String house5 = toString();
        houseAdd = cardVal();
        if (houseAdd == 11){
            if (houseHand > 11){
                houseAdd = houseAdd - 10;
            }
        }
        //getting image of card
        h5 = new Image((new FileInputStream("pics/"+house5+".png")));
        //setting image of card;
        //setting image of card
        houseCard5.setImage(h5);
        houseHand = houseHand + houseAdd;
        houseScore.setText(String.valueOf(houseHand));
        results();
    }
}
}
public void results(){

```

```

        if (playerHand > 21) {
            bust.setVisible(true);
            loseCounter++;
            gameInProgress = false;
            gameLogic();
            lose.setText(String.valueOf(loseCounter));
        }
        else if (houseHand == playerHand) {
            push.setVisible(true);
            gameInProgress = false;
            gameLogic();
        }
        else if (houseHand == 21 || houseHand > playerHand && houseHand < 21) {
            gameLose.setVisible(true);
            loseCounter++;
            gameInProgress = false;
            gameLogic();
            lose.setText(String.valueOf(loseCounter));
        }
        else if (playerHand == 21 && playerCardCount == 2) {
            blackJack.setVisible(true);
            gameInProgress = false;
            gameLogic();
            winCounter = winCounter + 2;
            win.setText(String.valueOf(winCounter));
        }
        else if (playerHand > houseHand && playerHand < 21 || houseHand > 21) {
            gameWin.setVisible(true);
            winCounter++;
            gameInProgress = false;
            gameLogic();
            win.setText(String.valueOf(winCounter));
        }
        if (loseCounter >= 3){
            gameOverAndDone();
        }
    }

    public void gameOverAndDone(){
        gameLose.setVisible(false);
        bust.setVisible(false);
        lgameOver.setVisible(true);
        gameInProgress = false;
        gameOver = true;
        gameLogic();
    }

    public void endGame(){
        playerCard1.setImage(null);
        playerCard2.setImage(null);
        playerCard3.setImage(null);
    }

```

```

        playerCard4.setImage(null);
        playerCard5.setImage(null);
        houseCard1.setImage(null);
        houseCard2.setImage(null);
        houseCard3.setImage(null);
        houseCard4.setImage(null);
        houseCard5.setImage(null);
        playerCardCount = 0;
        houseCardCount = 0;
        playerHand = 0;
        houseHand = 0;

    }

    public void hideGameStats(){
        bust.setVisible(false);
        gameWin.setVisible(false);
        gameLose.setVisible(false);
        push.setVisible(false);
        blackJack.setVisible(false);
        lgameOver.setVisible(false);
    }

    public String suit(){
        String suit = null;
        int max = 4;
        int min = 1;
        int range = max - min + 1;
        int suitNum = (int)(Math.random()*range+min);
        switch (suitNum) {
            case 1 -> suit = "CLUBS";
            case 2 -> suit = "HEARTS";
            case 3 -> suit = "DIAMONDS";
            case 4 -> suit = "SPADES";
        }
        return suit;
    }

    public String card(){
        String card = null;
        int max = 13;
        int min = 1;
        int range = max - min + 1;
        cardNum = (int)(Math.random()*range+min);
        switch (cardNum) {
            case 1 -> card = "TWO";
            case 2 -> card = "THREE";
            case 3 -> card = "FOUR";
            case 4 -> card = "FIVE";
            case 5 -> card = "SIX";

```

```

        case 6 -> card = "SEVEN";
        case 7 -> card = "EIGHT";
        case 8 -> card = "NINE";
        case 9 -> card = "TEN";
        case 10 -> card = "JACK";
        case 11 -> card = "QUEEN";
        case 12 -> card = "KING";
        case 13 -> card = "ACE";
    }
    return card;
}

public int cardVal(){
    int cardVal = 0;
    int value = cardNum;
    switch (value) {
        case 1 -> cardVal = 2;
        case 2 -> cardVal = 3;
        case 3 -> cardVal = 4;
        case 4 -> cardVal = 5;
        case 5 -> cardVal = 6;
        case 6 -> cardVal = 7;
        case 7 -> cardVal = 8;
        case 8 -> cardVal = 9;
        case 9, 10, 11, 12 -> cardVal = 10;
        case 13 -> cardVal = 11;
    }
    return cardVal;
}

public String toString(){
    return card() + " OF " + suit();
}

public void sortByScore(){
    int size = scores.size();
    int end = size-1;
    boolean sorted = false;
    for (int i = 0; i < size-1 && !sorted; i++){
        sorted = true;
        for (int j=0; j<end; j++){
            HighScores y1 = (HighScores) scores.get(j);
            HighScores y2 = (HighScores) scores.get(j+1);
            if (y1.getScore()< y2.getScore()){
                sorted = false;
                scores.remove(j);
                scores.add(j+1, y1);
            }
        }
        end--;
    }
}

```

```

        public static void main(String[] args) {
            launch();
        }
    }
}

```

High Scores Class:

```

package com.example.blackjack;

import java.util.Comparator;

public class HighScores {
    private String name;
    private int score;

    @Override
    public String toString() {
        return name + "," + score;
    }

    public HighScores(String name, int score) {
        this.name = name;
        this.score = score;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getScore() {
        return score;
    }

    public void setScore(int score) {
        this.score = score;
    }
}

```

My Custom ArrayList:

```

package com.example.blackjack;

public class MyArrayList implements MyList{
    Object[] obj =new Object[1];
}

```

```

int size=0;
public int size(){
    return size;
}

public Object get(int index) {
    if (index < 0 || index >= size){
        throw new IndexOutOfBoundsException();
    }
    return obj[index];
}

public void add(Object o){
    if(size==obj.length){
        Object[] obj2 = new Object[obj.length*2+1];
        for (int i=0; i<obj.length; i++){
            obj2[i]=obj[i];
        }
        obj=obj2;
    }
    obj[size]=o;
    size++;
}

public void add(int index, Object o) {
    if (index < 0 || index > size)
        throw new IndexOutOfBoundsException();
    Object[] obj2;
    if (size == obj.length) {
        obj2 = new Object[obj.length * 2 + 1];
        for (int i = 0; i < obj.length; i++) {
            obj2[i] = obj[i];
        }
        obj = obj2;
    }
    for (int i = obj.length-1; i > index; i--) {
        obj[i] = obj[i - 1];
    }
    obj[index] = o;
    size++;
}

public void remove(int index) {
    if (index < 0 || index >= size) {
        System.out.println("Index " + index + " is out of Bounds.");
        throw new IndexOutOfBoundsException();
    }
    else{
        for (int i = 0; i < size - 1; i++){

```

```

        obj[i] = obj [i+1];
    }
    size--;
}

}

public void remove (Object o){
    int index=indexOf(o);
    if(index>-1)
        remove(index);
}

public void set (int index, Object o){
    if (index < 0 || index >= size)
        throw new IndexOutOfBoundsException();
    obj[index]=o;
}

public boolean contains (Object o){
    return indexOf(o)>-1;
}

public int indexOf (Object o){
    for (int i=0; i<size; i++){
        if(obj[i]==o)
            return i;
    }
    return -1;
}

public boolean isEmpty(){
    return size==0;
}

public void removeAll(){
    obj=new Object[1];
    size=0;
}

}

```

My File Class:

```

package com.example.blackjack;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;

```

```

import java.io.PrintWriter;
import java.util.ArrayList;

public class MyFile {
    public static ArrayList<String> readFile(String fname) {

        ArrayList<String> myList = new ArrayList<>();
        try {
            File file = new File(fname);
            BufferedReader br = new BufferedReader(new FileReader(file));
            String st;

            while ((st = br.readLine()) != null) {
                myList.add(st);
            }
            br.close();
        } catch (Exception e) {
            e.printStackTrace();
        }

        return myList;
    }

    public static void writeFile(ArrayList<HighScores> list, String fname) {
        try{
            File file= new File(fname);
            PrintWriter pw = new PrintWriter(file);
            for( int i=0; i<list.size(); i++){
                pw.println(list.get(i));
            }
            pw.close();
        }catch (Exception e){e.printStackTrace();}

    }

    public static void writeFile1(MyArrayList list, String fname) {
        try{
            File file= new File(fname);
            PrintWriter pw = new PrintWriter(file);
            for( int i=0; i<list.size(); i++){
                pw.println(list.get(i));
            }
            pw.close();
        }catch (Exception e){e.printStackTrace();}

    }
}

```

```

package com.example.blackjack;

public interface MyList {
    int size();
    Object get(int index);
    void add(Object o);
    void add(int index, Object o);
    void remove(int index);
    void remove (Object o);
    void set (int index, Object o);
    boolean contains (Object o);
    int indexOf (Object o);
    boolean isEmpty();
    void removeAll();
}

package com.example.blackjack;

public class Records {
    private int winRecord;

    @Override
    public String toString() {
        return String.valueOf(winRecord);
    }

    public Records(int winRecord) {
        this.winRecord = winRecord;
    }

    public int getWinRecord() {
        return winRecord;
    }

    public void setWinRecord(int winRecord) {
        this.winRecord = winRecord;
    }
}

package com.example.blackjack;

import javafx.scene.Parent;

public class Card extends Parent {
    public String suit(){
        String suit = null;
        int max = 4;
        int min = 1;
        int range = max - min + 1;
        int suitNum = (int)(Math.random()*range+min);
    }
}

```

```

        switch (suitNum) {
            case 1 -> suit = "CLUBS";
            case 2 -> suit = "HEARTS";
            case 3 -> suit = "DIAMONDS";
            case 4 -> suit = "SPADES";
        }
        return suit;
    }
}

public String card(){
    String card = null;
    int max = 12;
    int min = 1;
    int range = max - min + 1;
    cardNum = (int)(Math.random()*range+min);
    switch (cardNum) {
        case 1 -> card = "TWO";
        case 2 -> card = "THREE";
        case 3 -> card = "FOUR";
        case 4 -> card = "FIVE";
        case 5 -> card = "SIX";
        case 6 -> card = "SEVEN";
        case 7 -> card = "EIGHT";
        case 8 -> card = "NINE";
        case 9 -> card = "TEN";
        case 10 -> card = "JACK";
        case 11 -> card = "QUEEN";
        case 12 -> card = "KING";
    }
    return card;
}

public int cardVal(){
    int cardVal = 0;
    int value = cardNum;
    switch (value) {
        case 1 -> cardVal = 2;
        case 2 -> cardVal = 3;
        case 3 -> cardVal = 4;
        case 4 -> cardVal = 5;
        case 5 -> cardVal = 6;
        case 6 -> cardVal = 7;
        case 7 -> cardVal = 8;
        case 8 -> cardVal = 9;
        case 9, 10, 11, 12 -> cardVal = 10;
    }
    return cardVal;
}

public String toString(){
    return card() + "OF" + suit();
}

```

```
    }  
  
    int cardNum;  
}
```